

Phase I - D1 – Design

Group Name: The Comp 100

Group Members:

Bariş Cem Bakay

Bera Nazlı

Ege Aksoy

Ege Çınar

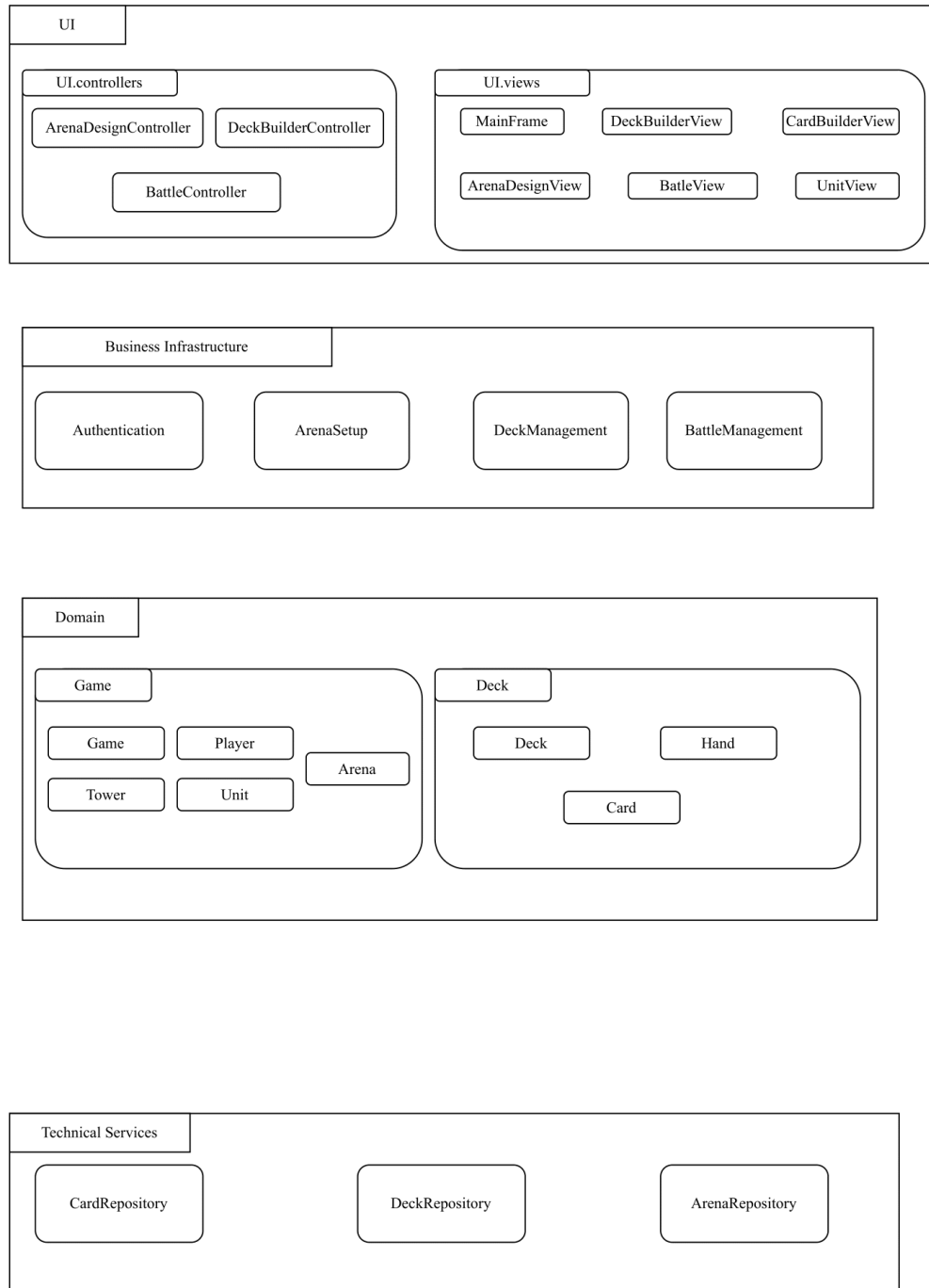
Burak Faiz

Burak Gerçekaslan

Table of Contents

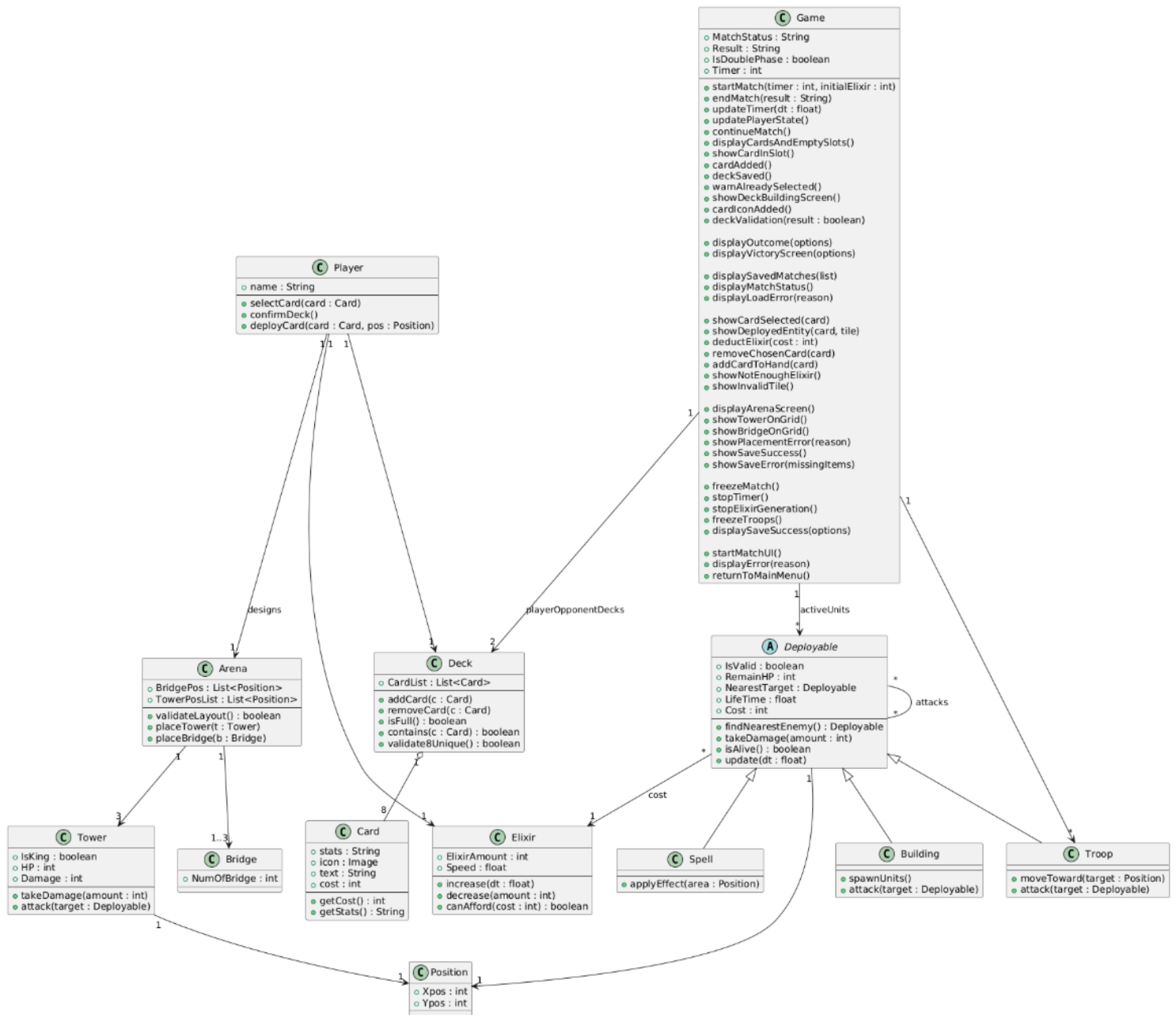
1) UML Package Diagram.....	3
2) UML Class Diagrams	4
3) Design Alternatives	5
4) Interaction Diagrams.....	8
4.1) Sequence Diagrams.....	8
4.2) Communication Diagrams.....	18

1) UML Package Diagram



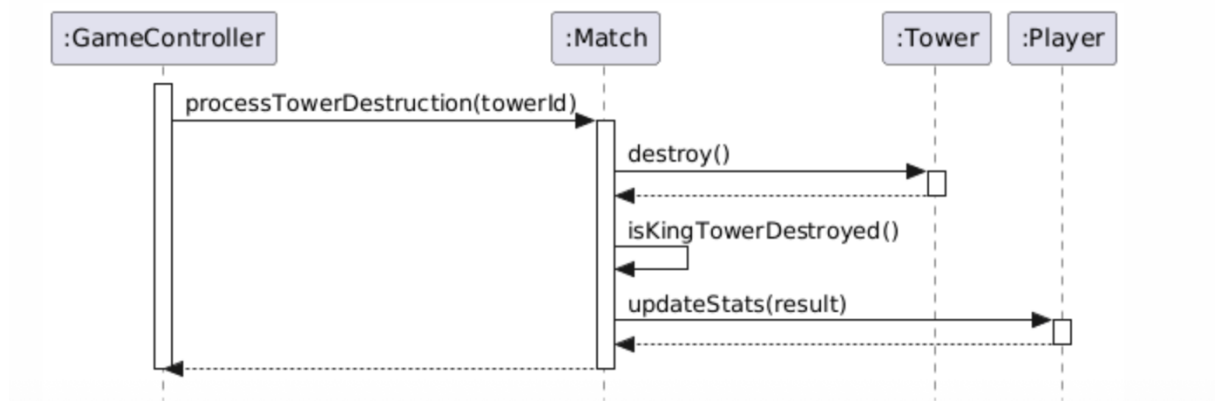
2) UML Class Diagrams

KURoyale Domain UML (with SSD-derived Methods)



3) Design Alternatives

Use Case 7: Win Match Alternative - Information Expert



In this design, instead of letting the GameController determine the winner, the controller simply triggers the win & check action. The Match object, already responsible for tracking players, towers, turns, and match state, handles the full logic of determining who wins

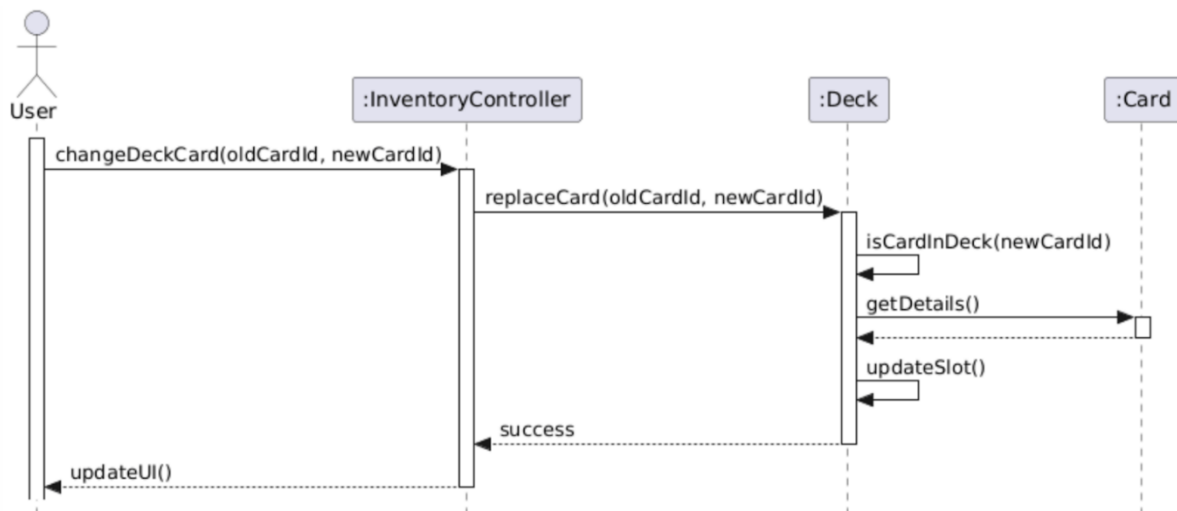
Pros

- High Cohesion: GameController becomes simpler. It focuses only on receiving input and passing it to the Match object. It does not need to know the rules of how to end a match.
- Encapsulation: All win condition rules are kept inside the Match class, which centralizes and organizes the logic. If win conditions need to be changed later, only Match will be modified, not the Controller. (Separation of Concerns)
- The domain model also becomes clearer, since the Match class is naturally responsible for determining the winner.

Cons

- Coupling: The Match class is more tightly coupled to the Player and the Tower class since it talks to them directly. A change in these classes (data type, function implementation, etc.) will enforce refactoring the Match class.
- Responsibility Overload: If excessive logic will be moved into Match, the class can violate the Single Responsibility Principle. It risks becoming an object that knows and does too much, adding more internal complexity inside the Match class.

Use Case 2: Build Deck Alternative - Information Expert & Low Coupling Principle



Instead of the Controller managing the removal and addition as two separate steps, logic can be moved into the Deck class as a single, atomic operation.

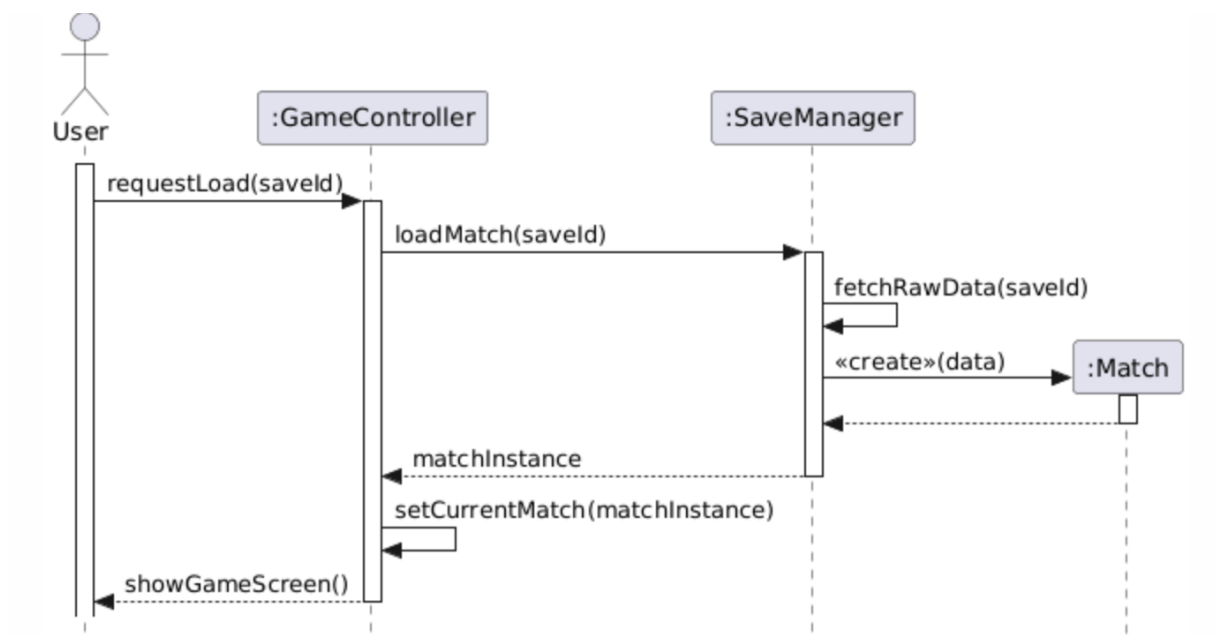
Pros

- **Data Integrity:** In the original design, there is a duration between `removeCard` and `addCard` where the Deck might be in invalid state. By using `replaceCard` the Deck transitions from one valid state to another instantly.
- **Atomicity:** By using `replaceCard`, the operation either succeeds or fails completely. The Deck never exists in a half finished state.
- **Low Coupling:** The `InventoryController` no longer needs to know the internal order of operations. It just sends a request.

Cons

- **Complicated Model:** The Deck class becomes larger because it now contains logic for swapping and validating, rather than just holding data. This can make the class heavier and harder to migrate to another framework that can expect different logic. Care must be taken to prevent the Deck from evolving into a “God object”.
- **Rigid Interface:** If the Controller wants to do something slightly different, like removing a card but not adding one, it cannot use the atomic `replaceCard` method.

Use Case 9: Reload Match Alternative - Pure Fabrication



This design alternative involves creating a helper class, SaveManager, that does not represent a real world object but exists to solve a technical problem. Instead of the controller loading and parsing saved game data, a new helper class is created.

Pros

- High Cohesion & Separation of Concerns: The GameController only handles user input, not file operations. The parsing data is hidden inside SaveManager. Separating saving and loading logic into a dedicated class significantly improves separation of concerns.
- Reusability: If match needs to be loaded from a different part of the software, SaveManager can be reused without duplicating the logic.

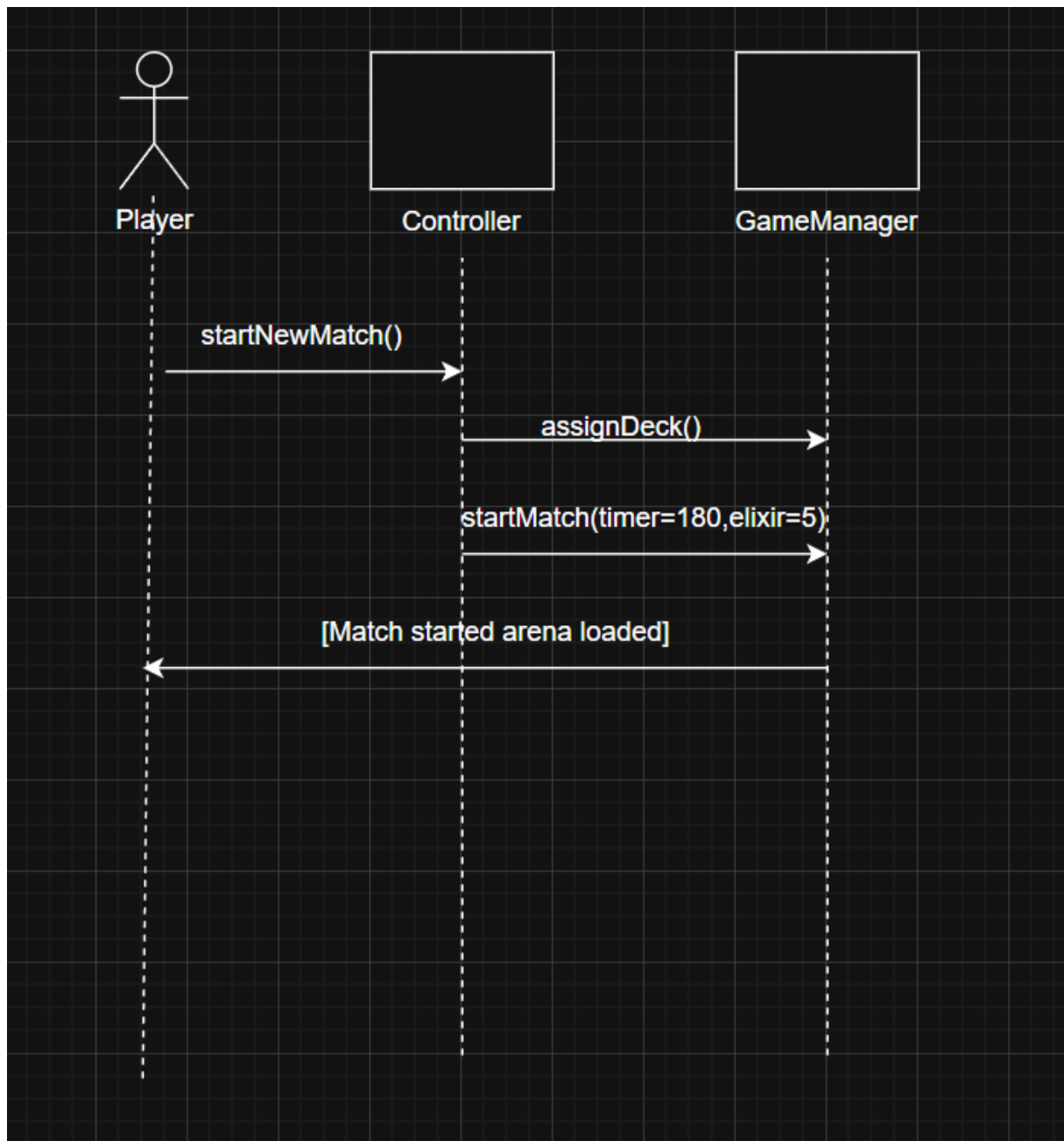
Cons

- Increased Class Amount: A new class, SaveManager, has been added to the system, increasing the overall number of classes to manage. Additional maintenance and testing effort is needed to ensure the save and load functions work correctly.
- Complex refactoring: If a new attribute is added to Player, SaveManager must be updated to use that new field. SaveManager separates the data from the saving logic, increasing the risk of them falling out of sync.

4) Interaction Diagrams

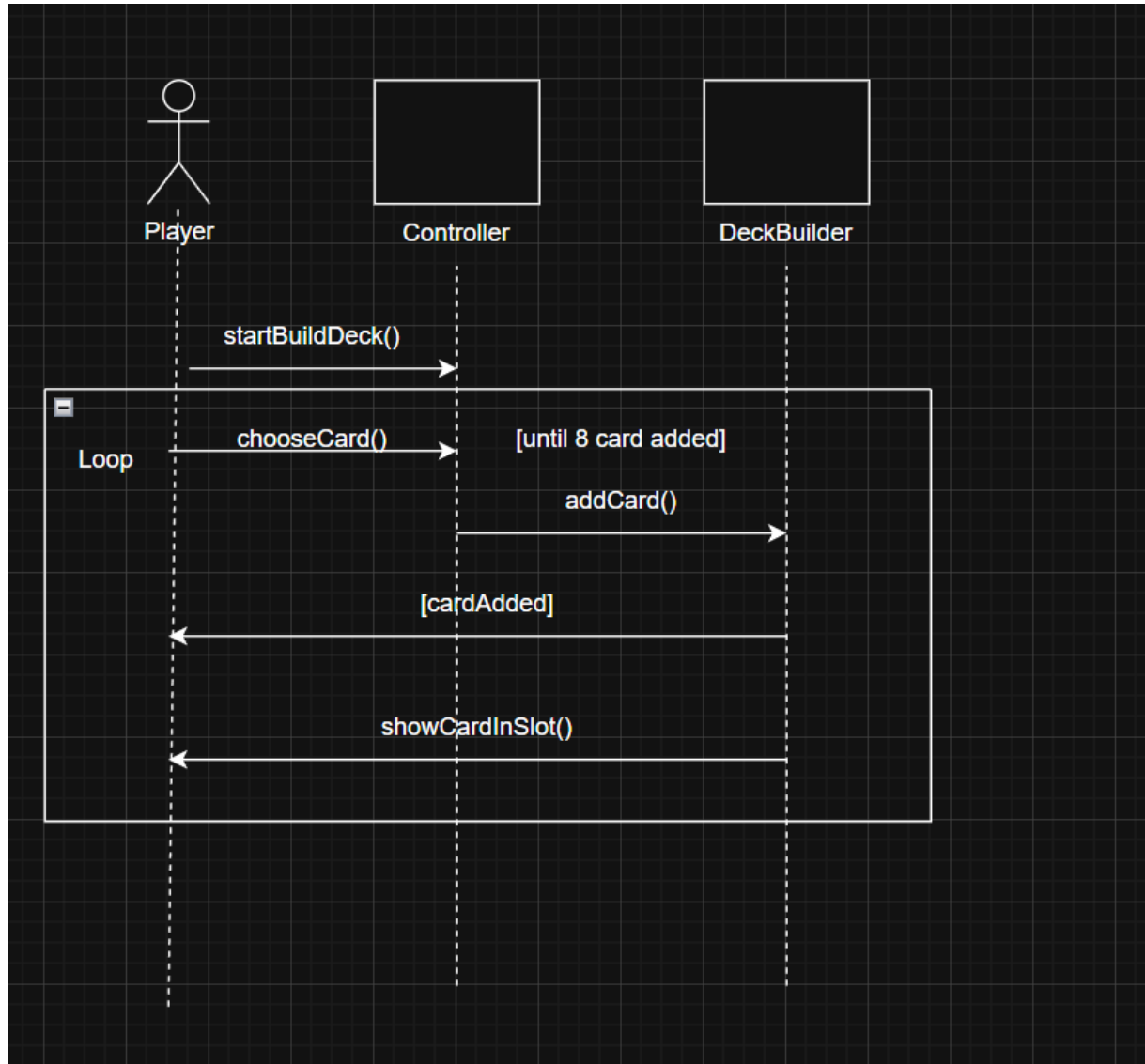
4.1) Sequence Diagrams

UC1) Start New Match

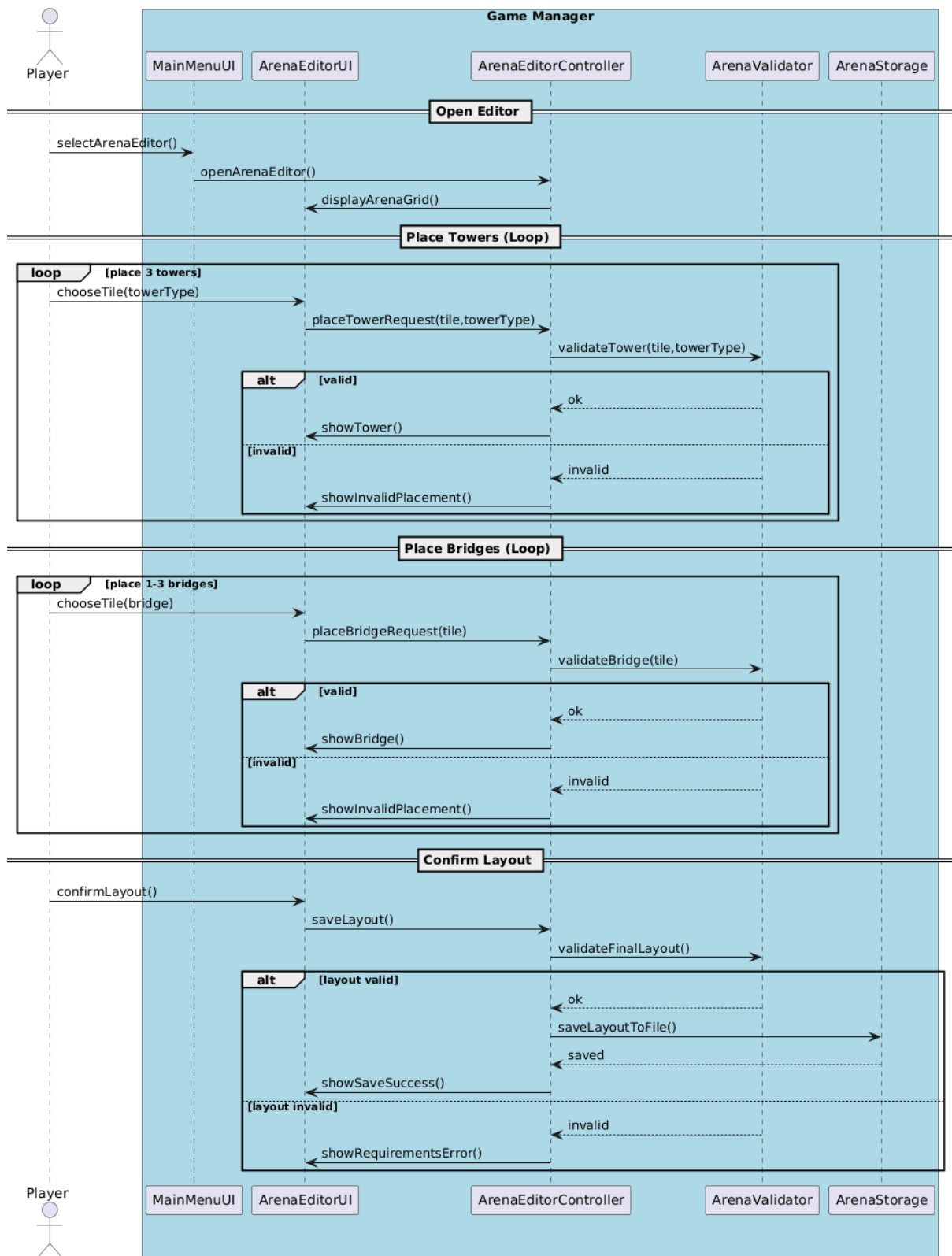



```
sequenceDiagram
    actor Player
    participant Controller
    participant DeckBuilder

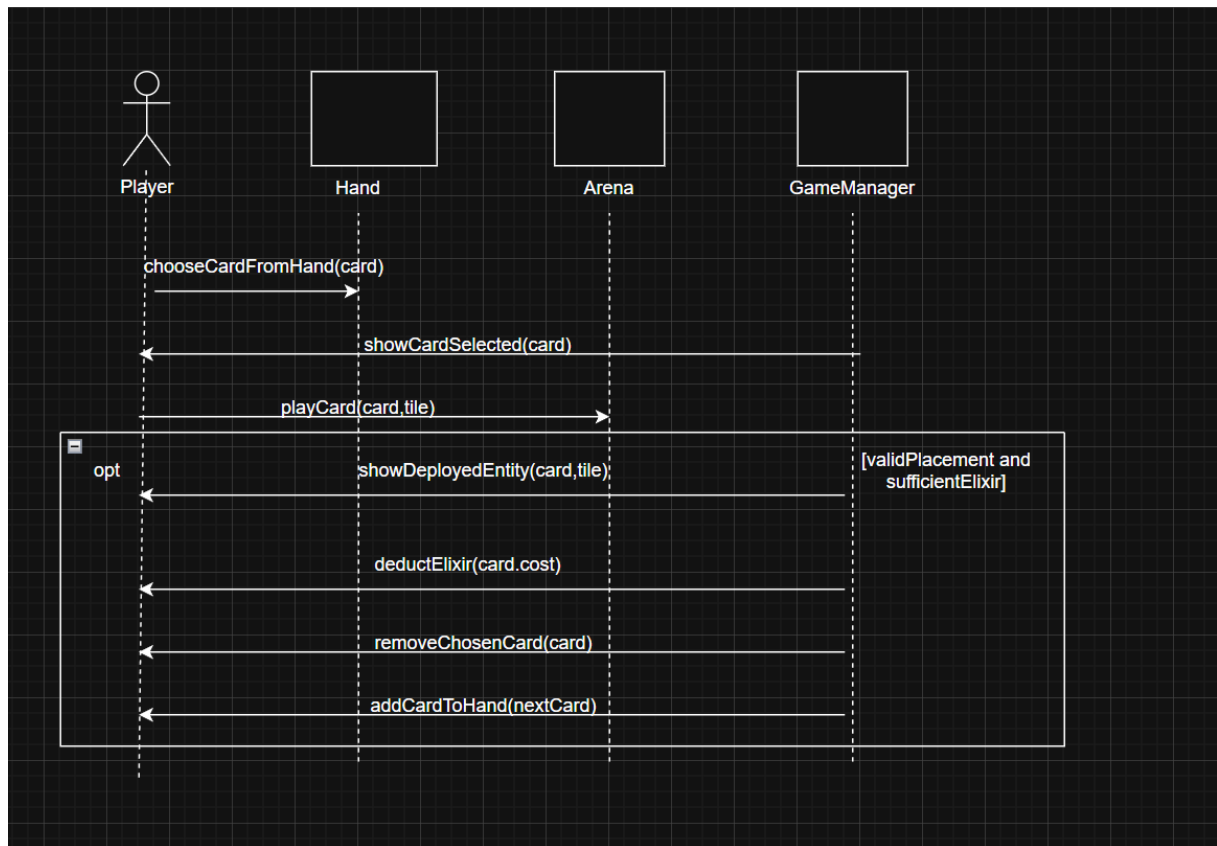
    Player->>Controller: startBuildDeck()
    activate Controller
    Note over Controller: Loop
    Controller->>Controller: chooseCard() [until 8 card added]
    Controller->>DeckBuilder: addCard()
    DeckBuilder-->>Player: [cardAdded]
    DeckBuilder-->>Player: showCardInSlot()
    deactivate DeckBuilder
    deactivate Controller
```



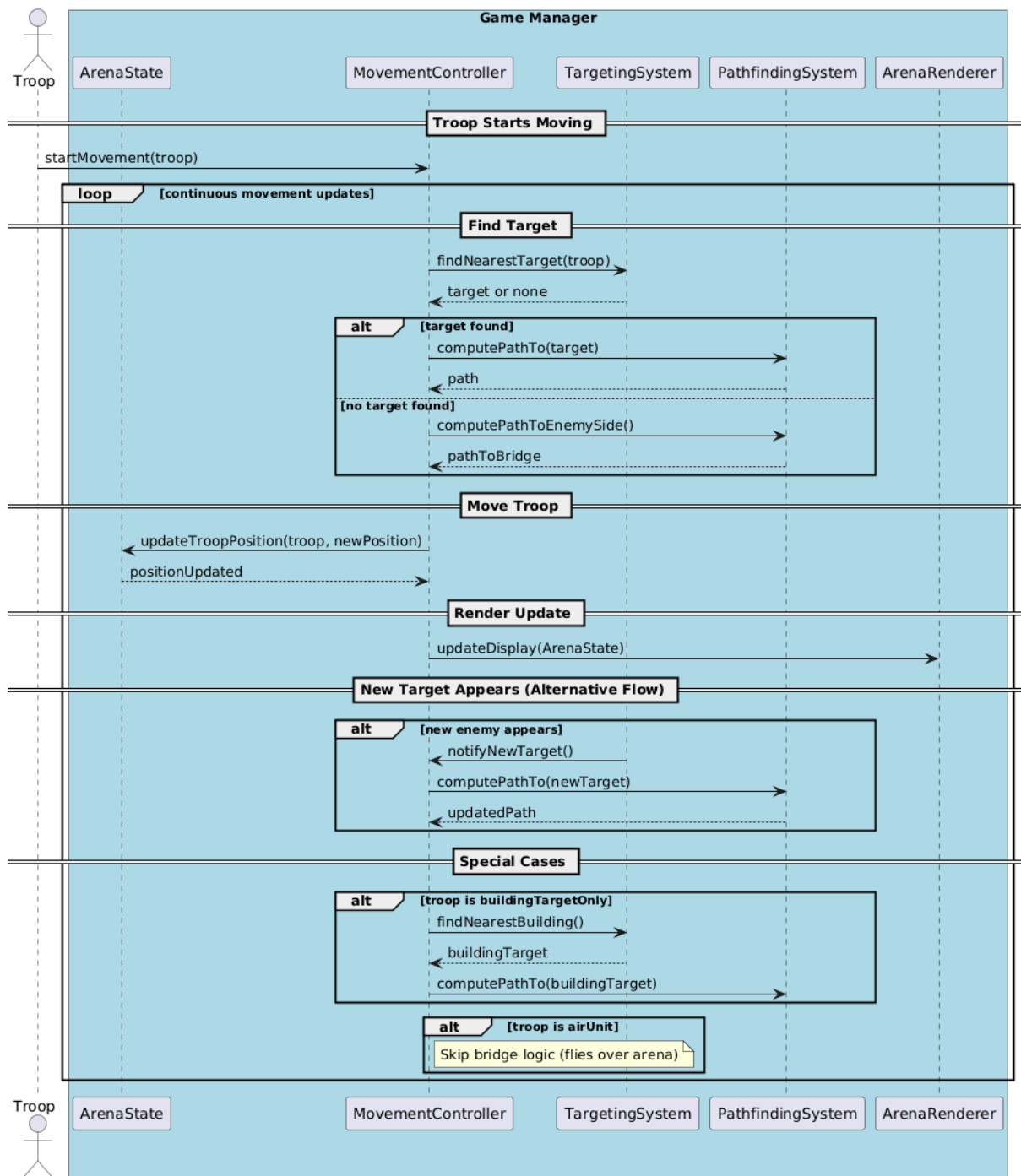
UC3) Design Arena



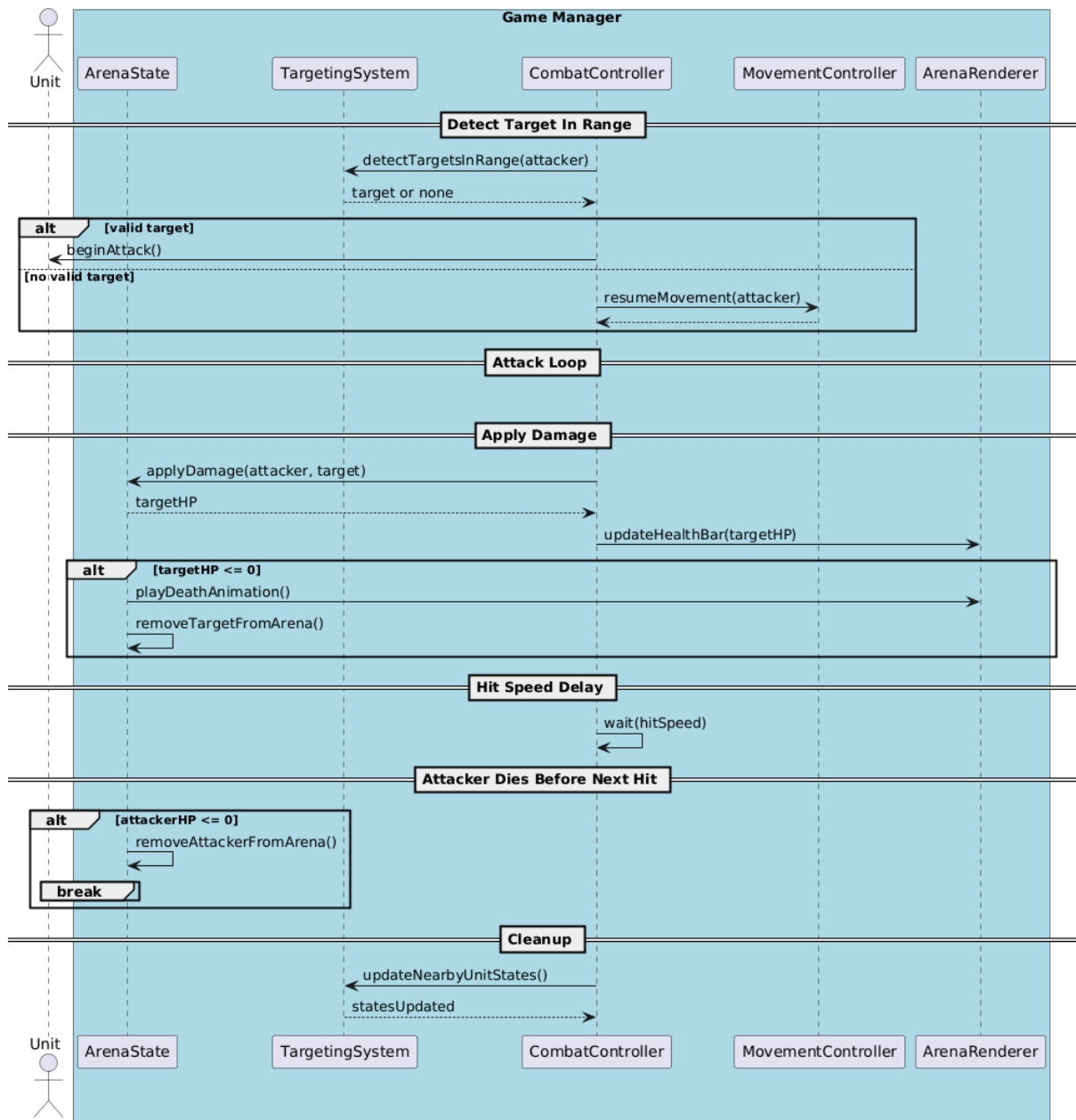
UC4) Deploy Card



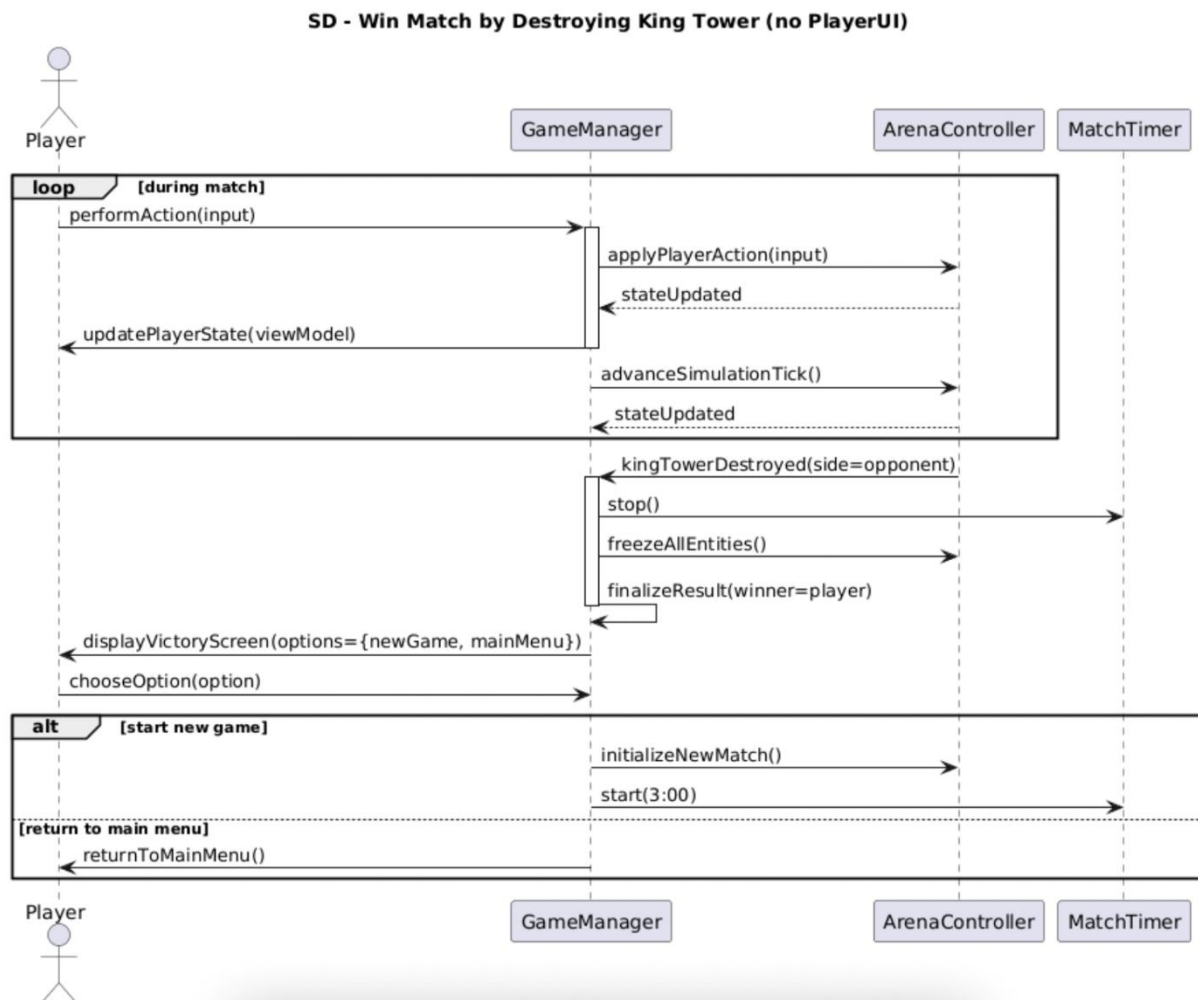
UC5) Move Troop



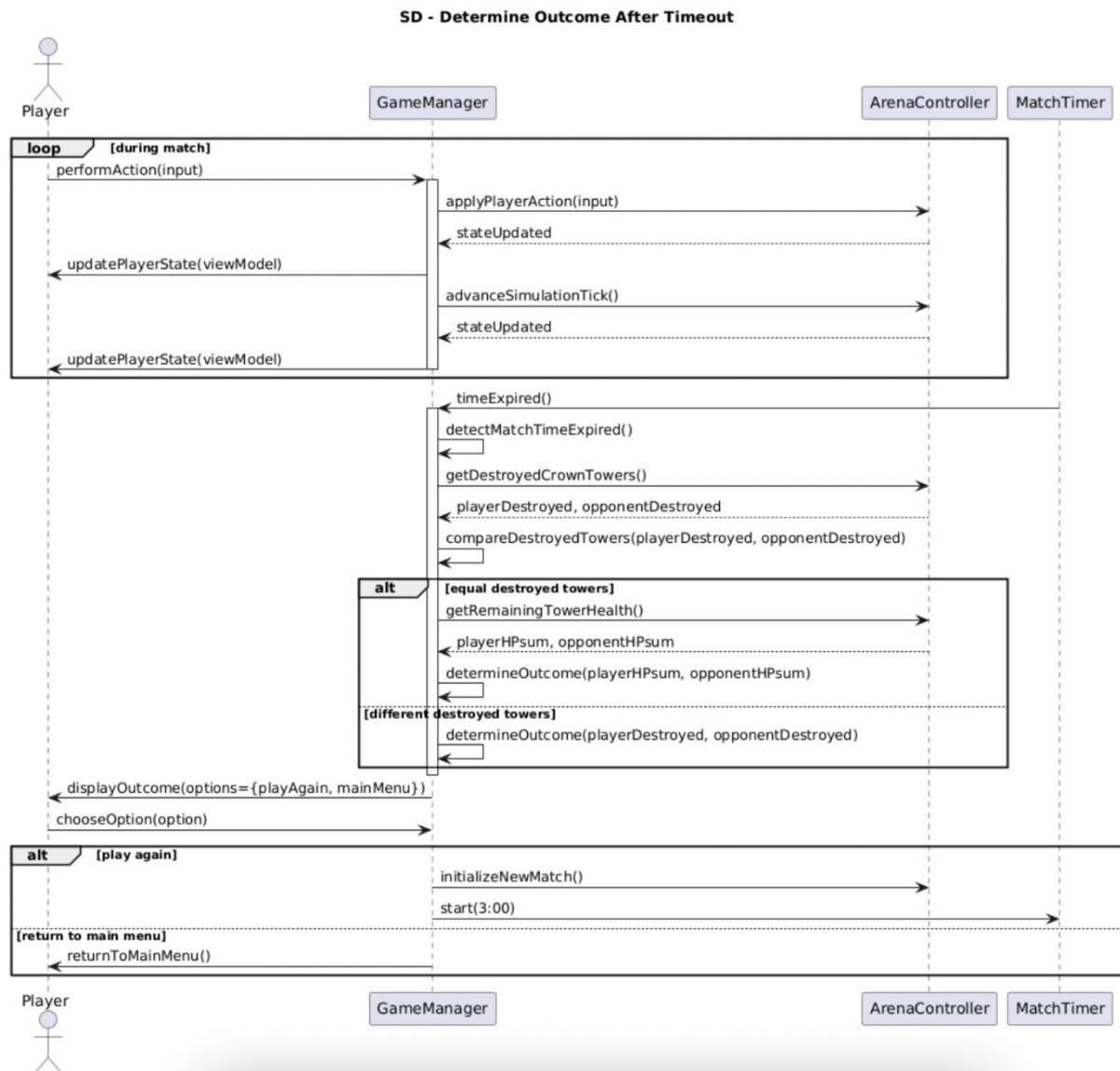
UC6) Combat



UC7) Win Match

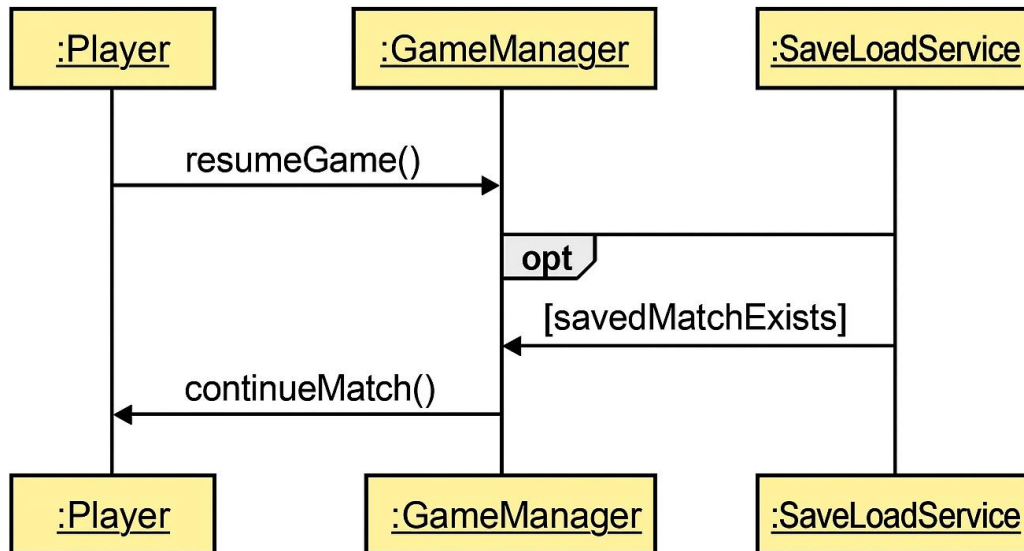


UC8) Determine Outcome



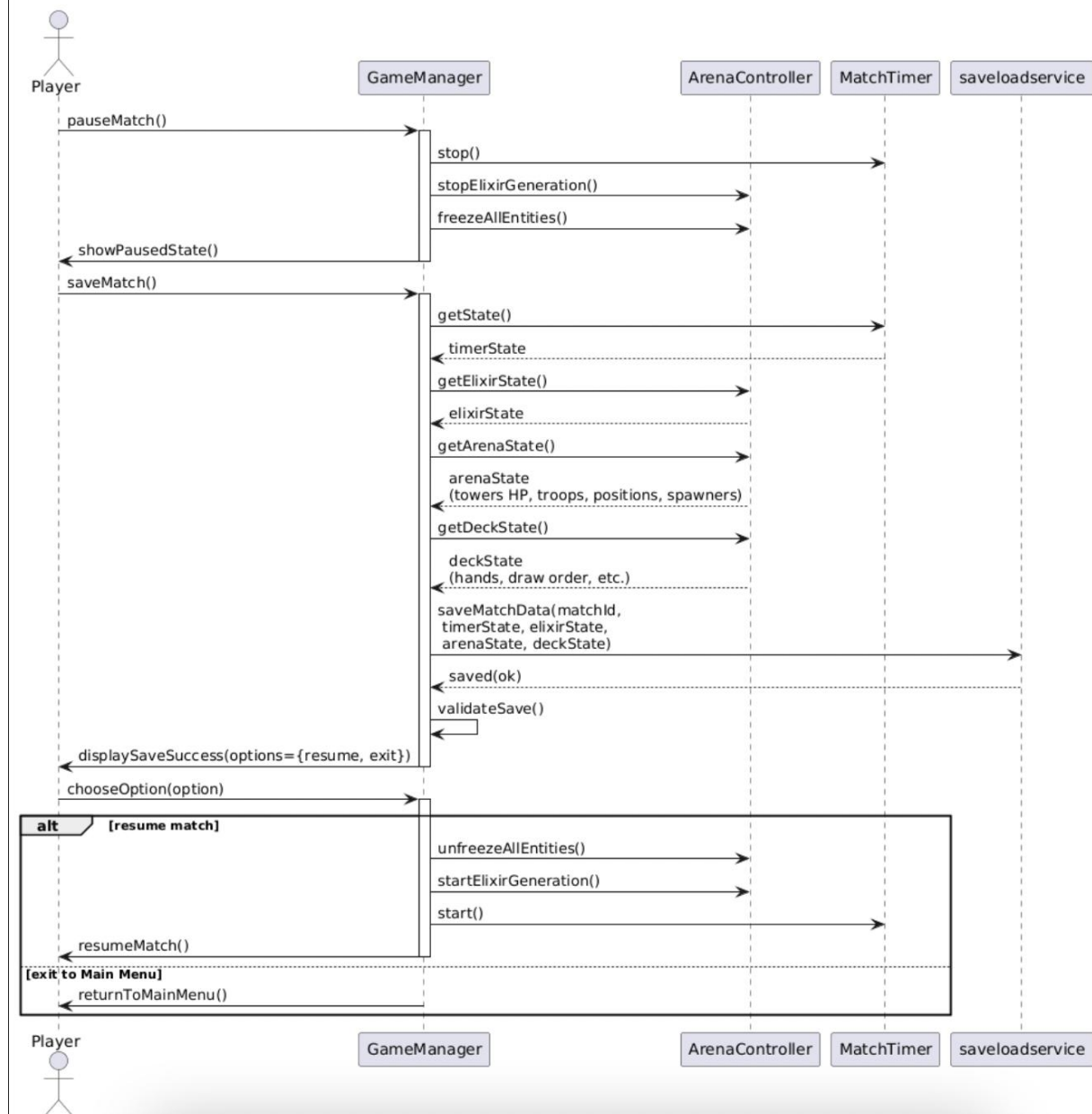
UC9)

Reload Match



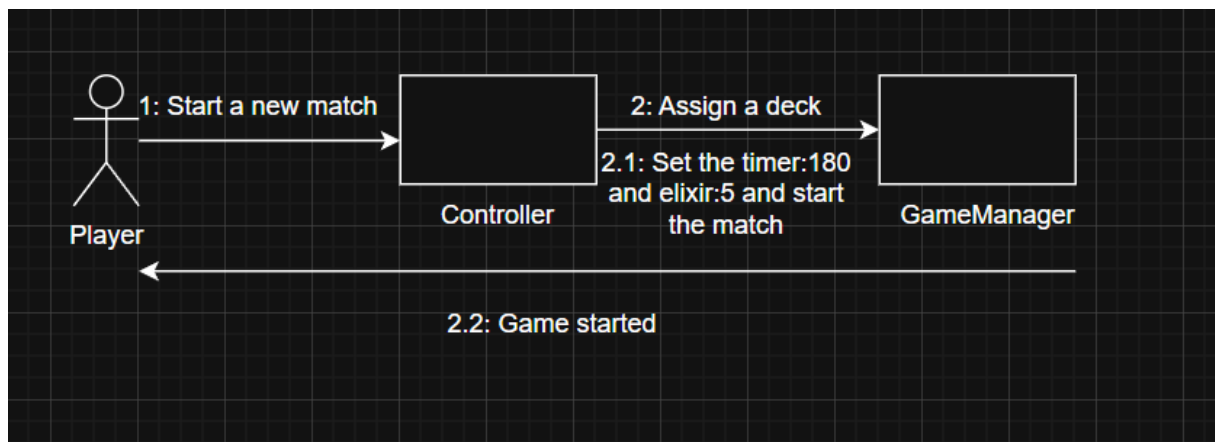
UC10) Save Match

SD - Save Match (Pause, Collect, Persist, Confirm)

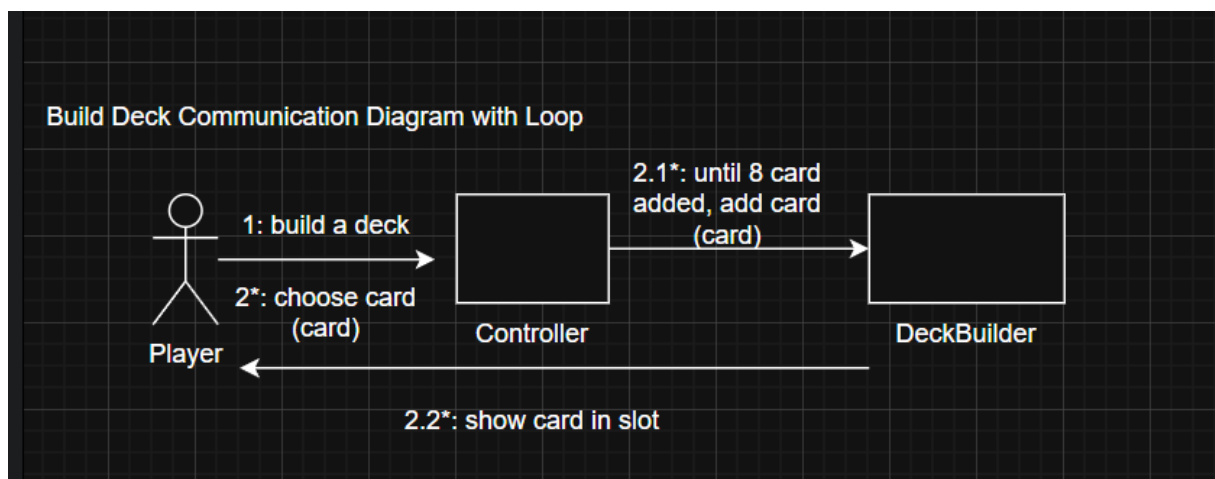


4.2) Communication Diagrams

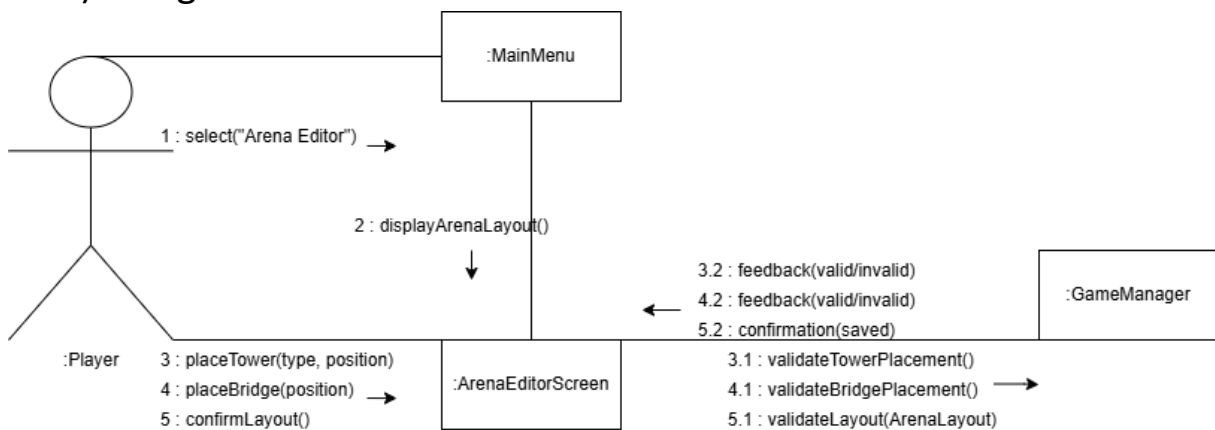
UC1) Start New Match



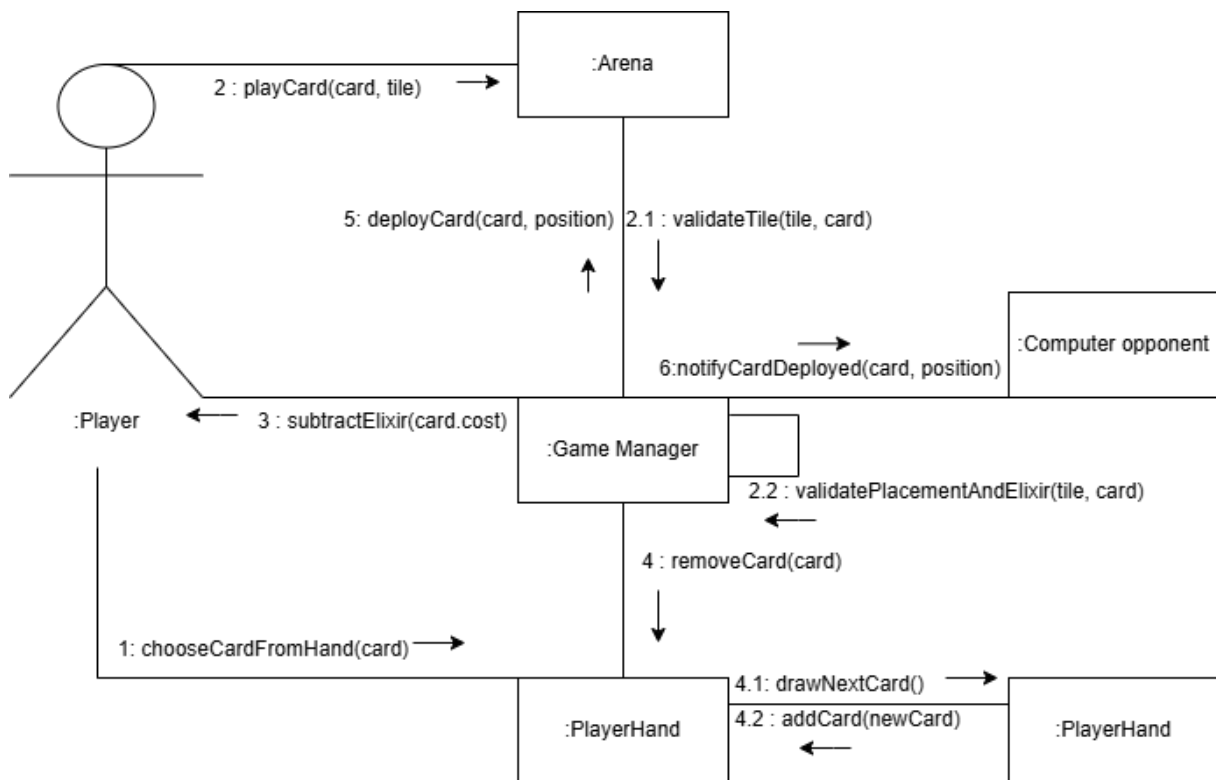
UC2) Build Deck



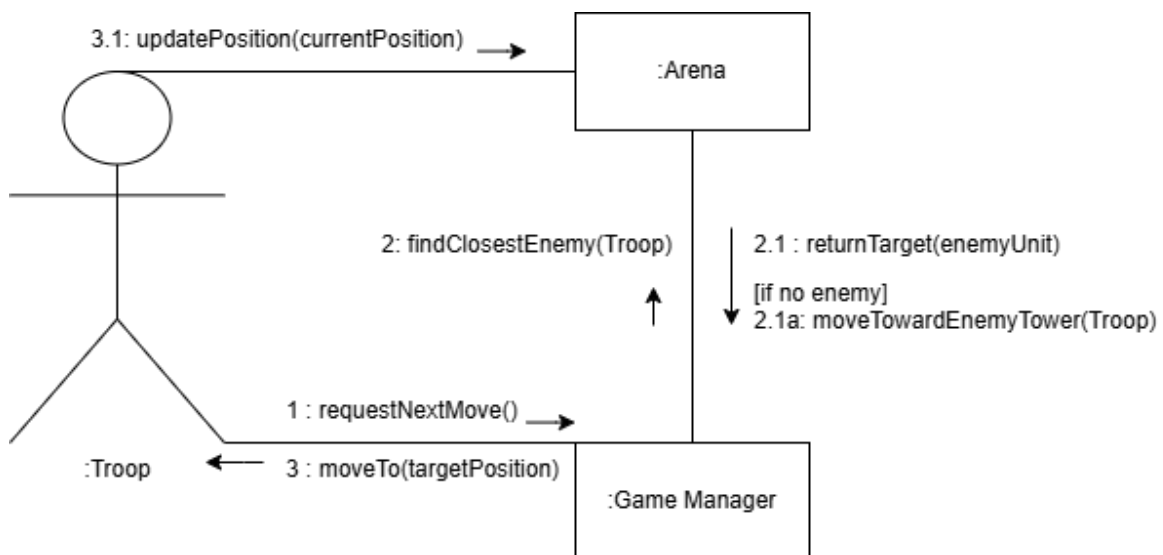
UC3) Design Arena



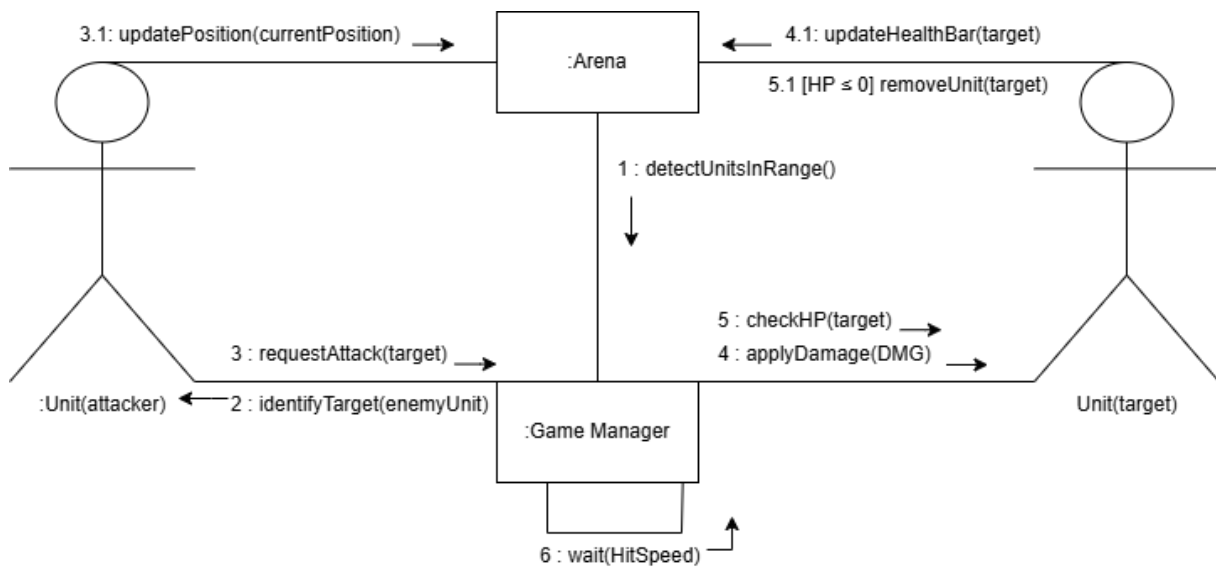
UC4) Deploy Card



UC5) Move Troop

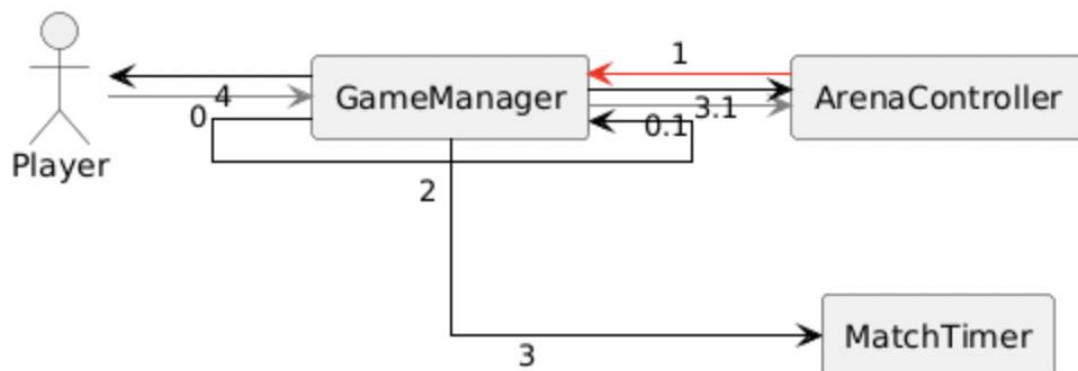


UC6) Combat



UC7) Win Match

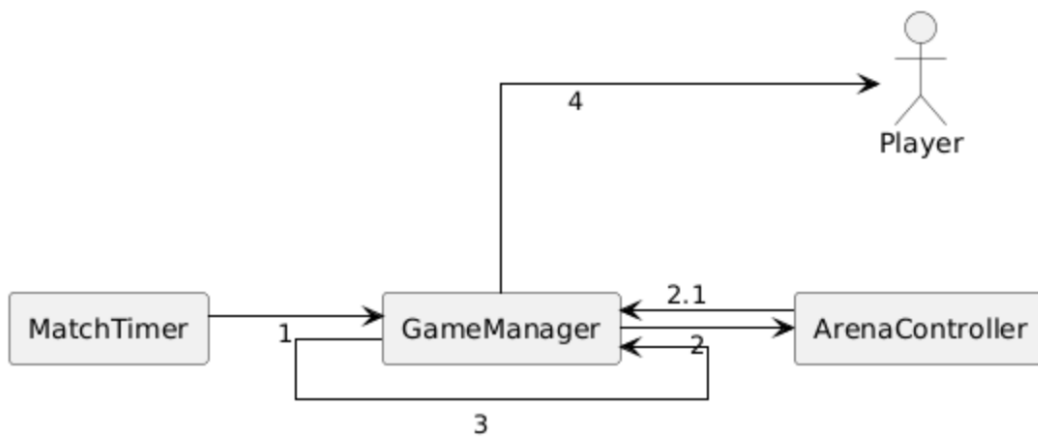
Communication Diagram - UC7 Win by King Tower



- 0,0.1 Ongoing match play (optional)
- 1 King Tower destroyed
 - 2 detectKingTowerDestruction()
 - 3 stop() + freezeAllEntities()
 - 4 displayVictoryScreen({newGame, mainMenu})

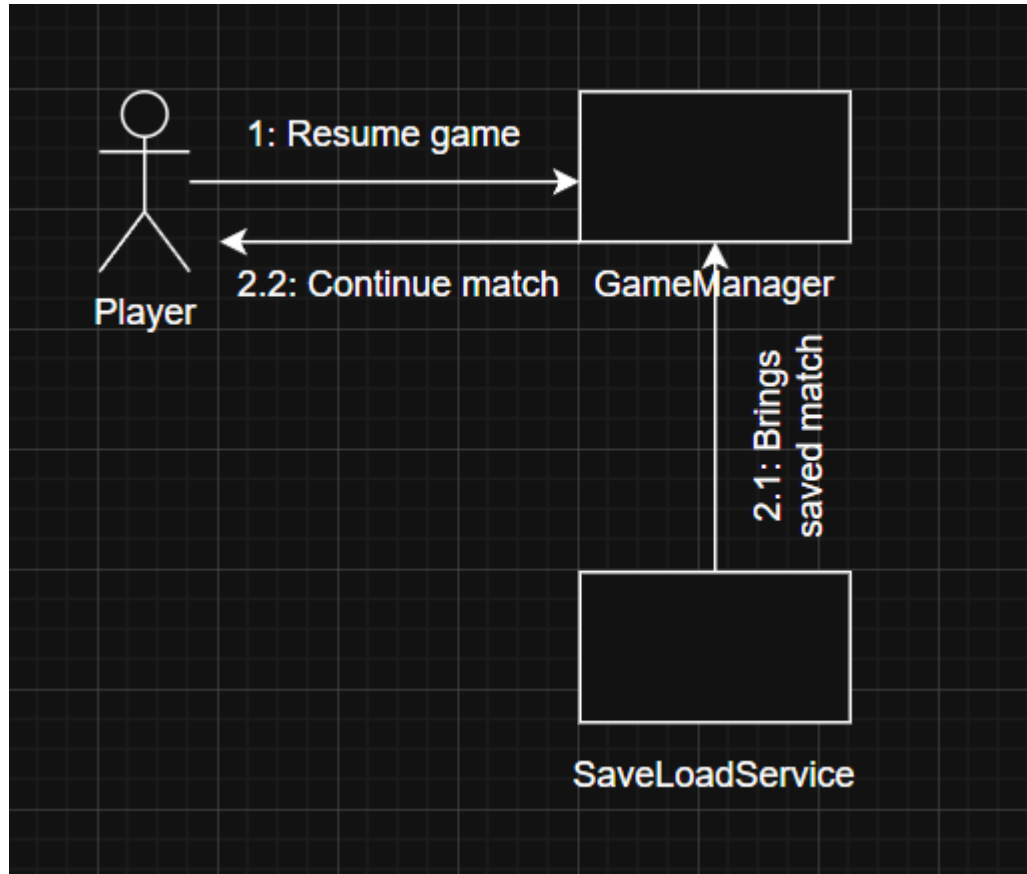
UC8) Determine Outcome

Communication Diagram - UC8 Timeout Outcome



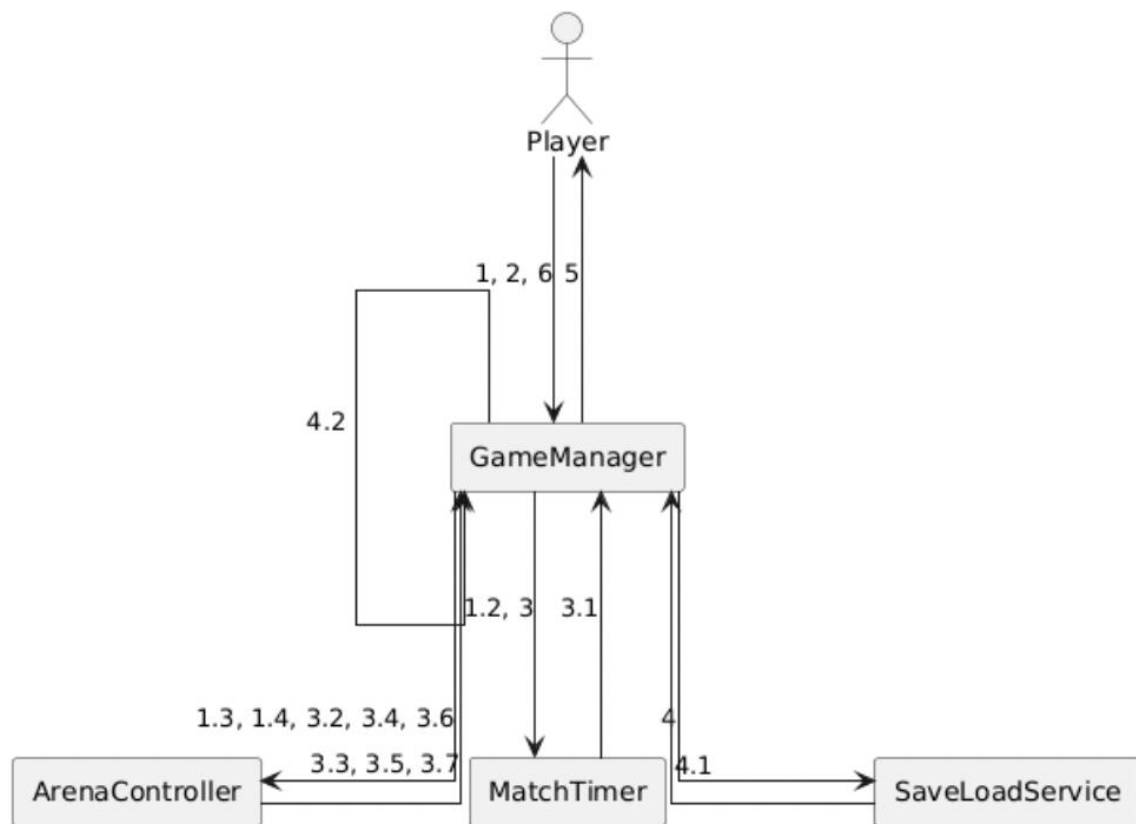
- 1 detectMatchTimeExpired()
- 2 getDestroyedCrownTowers()
- 2.1 playerDestroyed, opponentDestroyed
- 3 determineOutcome() (player wins if more destroyed; else loses)
- 4 displayOutcome({playAgain, mainMenu})

UC9) Reload Match



UC10) Save Match

Communication Diagram - UC10 Save Match



- 1 Player→GM: pauseMatch()
- 1.2 GM→T: stop()
- 1.3 GM→AC: stopElixirGeneration()
- 1.4 GM→AC: freezeAllEntities()
- 2 Player→GM: saveMatch()
- 3 GM→T: getState()
- 3.1 T→GM: timerState
- 3.2 GM→AC: getElixirState()
- 3.3 AC→GM: elixirState
- 3.4 GM→AC: getArenaState()
- 3.5 AC→GM: arenaState
- 3.6 GM→AC: getDeckState()
- 3.7 AC→GM: deckState
- 4 GM→SLS: saveMatchData(matchId, timerState, elixirState, arenaState, deckState)
- 4.1 SLS→GM: saved(ok)
- 4.2 GM→GM: validateSave()
- 5 GM→Player: displaySaveSuccess({resume, exit})
- 6 Player→GM: chooseOption(option)